

SQL QUESTION

1. Employees with More Than One Manager

Problem Statement: Find the employees who have more than one manager.

Solution: This query groups the table by the employee's ID and name, filtering for those where the count of distinct managers is greater than one.

```
SELECT employee_id, employee_name
FROM employee_managers
GROUP BY employee_id, employee_name
HAVING COUNT(DISTINCT manager_id) > 1;
```

2. Rank Employees from Different Departments

Problem Statement: Write a SQL query to rank employees within their respective departments based on their salaries.

Solution: This uses a window function to assign a rank to employees partitioned by their department.

```
SELECT department_id, employee_name, salary,
       DENSE_RANK() OVER(PARTITION BY department_id ORDER BY salary DESC) as
rank
FROM employees;
```

3. Users with Multiple Page Clicks

Problem Statement: Given a table called user_clicks, write a query to find users who have clicked on more than one distinct page.

Solution: This query selects the user ID, groups the results, and filters for users with more than one unique page ID.

```
SELECT user_id
FROM user_clicks
GROUP BY user_id
HAVING COUNT(DISTINCT page_id) > 1;
```

4. Daily Song Plays Per User

Problem Statement: We have a table called song_plays that tracks each time a user plays a song. Write a query to return the number of songs played on each date for each user.

Solution: Group the data by user and date, utilizing COUNT(*) to calculate the total plays per day.

```
SELECT user_id, play_date, COUNT(*) as songs_played
FROM song_plays
GROUP BY user_id, play_date;
```

5. Neighborhoods with Zero Users

Problem Statement: We are given a users table and a neighborhoods table. Write a query that returns all neighborhoods with 0 users.

Solution: Perform a LEFT JOIN from the neighborhoods to the users table and filter for rows where the user ID is NULL.

SQL

```
SELECT n.neighborhood_name FROM neighborhoods n LEFT JOIN users u ON  
n.neighborhood_id = u.neighborhood_id WHERE u.user_id IS NULL;
```

6. Active Pages with the Latest Event

Problem Statement: Write a query to count the number of active pages (status = 'active') that have the latest event date among all events for each page. **Solution:** This utilizes a correlated subquery with NOT EXISTS to check if there are no other events for the same page with a later date.

```
SELECT COUNT(page_id)  
FROM pages p1  
WHERE status = 'active'  
AND NOT EXISTS (  
    SELECT 1 FROM pages p2  
    WHERE p1.page_id = p2.page_id  
    AND p2.event_date > p1.event_date  
);
```

7. Find Duplicate Rows

Problem Statement: How do you find duplicate rows in a table based on a specific column? **Solution:** Select the column, use GROUP BY, and apply a HAVING COUNT(*) > 1 filter to identify groups that appear more than once.

```
SELECT column_name, COUNT(*)  
FROM table_name  
GROUP BY column_name  
HAVING COUNT(*) > 1;
```